

Python Quizmaster - Project Overview

1. Project Workflow

The Python Quizmaster application follows a multi-screen event management workflow:

- Dashboard (Hub): The central entry point where the Quizmaster can monitor system status and navigate to different modules.
- Setup Phase: The Quizmaster uses 'Manage Teams' to set up participants, 'Question Bank' to curate questions, and 'Settings' to configure event rules (rounds, timers).
- Execution Phase (Control Center): During the live event, the Quizmaster operates the Control Center to push questions, reveal answers, and award points.
- Presentation Phase (Projector Screen): A separate browser window/screen displays the 'Projector Screen' to the audience. It reflects the live state (current question, timer, podium) in real-time based on the Quizmaster's actions.

2. Tools and Technologies Used

- React 19: Core frontend library for building the user interface.
- Vite: Extremely fast build tool and development server.
- React Router DOM: For handling client-side routing between the Dashboard, Control Center, and Projector Screen.
- Lucide React: Providing modern, crisp vector icons for the UI.
- Framer Motion: Used for complex animations, particularly in the Podium Ceremony and Projector Screen transitions.
- Canvas Confetti: For celebratory visual effects when revealing winners.
- PrismJS: Syntax highlighting for Python code snippets within quiz questions.
- CSS/Glassmorphism: Custom CSS styling utilizing frosted-glass effects (glass-panel) and CSS variables for theming.

3. Logics & Architecture (State Management)

The core logic relies on centralized state management, enabling synchronization across different

Python Quizmaster - Project Overview

views:

- Context API (QuizContext): A global state provider wraps the application. It holds the source of truth for the entire quiz event (questions, teams, current round, scores, active question).
- Actions & Reducers: The Context exposes an 'actions' object (e.g., nextQuestion, addScore, toggleTheme) that standardizes state mutations.
- Cross-Window Synchronization: By leveraging React Context (or LocalStorage events if spanning multiple actual windows), the Control Center's state changes instantly trigger re-renders on the Projector Screen.
- Component Modularity: The architecture separates layout (Dashboard) from feature-specific components (QuizWaitingScreen, PodiumCeremony), keeping the codebase maintainable.